VW after the capacity cuts: data and exhibits

Working title. Capacity Cuts Bought Time, Not Competitiveness: VW's Next Move on Unit Cost

This notebook builds every number and chart used in the 12-minute presentation. Nothing here is modelled: each figure is either calculated from the Eurostat file or typed in from a named, dated source.

Slide	Exhibit (saved in `data/exhibits/`)	Built in section
1	`01_margin_gap.png`	3.1
2	`02_import_trend.png`	3.2
3	`03_price_trend.png`	3.3
6	`06_savings_vs_margin.png`	3.4

How to run.

1. Put the Eurostat file `Comext_final.csv` in `data/raw/comext/` (section 1 lists the download settings).
2. Choose **Kernel, Restart Kernel and Run All Cells**. It takes a few seconds.
3. Collect the charts from `data/exhibits/` (each is saved twice: the full chart, and a `_slide` version without headline or source note for the deck). `key_numbers.csv` in the same folder lists every number quoted in the talk, with its source.

Rules followed.

- Raw files are never edited; cleaning happens in code and the cleaned tables go to `data/processed/`.
- Hand-entered inputs live in `data/inputs/` and carry a source and a retrieval date.
- Every chart headline is generated from the data, so a headline cannot disagree with its chart.

0. Setup

All settings sit in this one cell. Change a setting here and the whole notebook follows.

In [1]:
```
from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd
from matplotlib.ticker import MultipleLocator

# ---- folders ------------------------------------------------------------------
RAW_COMEXT, INPUTS = Path("data/raw/comext"), Path("data/inputs")
PROCESSED, EXHIBITS = Path("data/processed"), Path("data/exhibits")
for folder in (RAW_COMEXT, INPUTS, PROCESSED, EXHIBITS):
    folder.mkdir(parents=True, exist_ok=True)

# ---- data-quality rule for the trade data (section 1) -------------------------
# A reporter-month row is "suspect" if a car is worth under EUR 2,000 or weighs under 300 kg, once it has 200+ cars
MIN_EUR_PER_CAR, MIN_KG_PER_CAR, MIN_UNITS = 2000, 300, 200

# ---- EU duties and the dates that define "before" and "after" -----------------
DUTY_DATE = pd.Timestamp("2024-10-30")      # definitive countervailing duties start
STANDARD_DUTY, BYD_DUTY = 0.10, 0.17        # 10% standard duty + 17% countervailing duty (BYD rate)
PERIOD_BEFORE = ("2023-10-01", "2024-09-01")   # the 12 months before the duty month
PERIOD_AFTER = ("2024-11-01", "2025-12-01")    # Nov 2024 to Dec 2025 (Oct 2024 is the transition month, left out)
PERIOD_LATEST_START = "2026-01-01"             # runs to the latest month in the file
DEFAULT_VAT = 0.19                             # German VAT, used to take VAT off list prices

# ---- chart style: one accent colour, greys, no gridlines ----------------------
ACCENT, GREY, INK, MUTED = "#008c82", "#d3dedd", "#002733", "#5f7a80"   # VW press-release palette: teal, light gre


def finish(fig, title, note, filename, top=0.93, bottom=0.05, note_size=8):
    """Save the chart twice and show it: the full version (headline + source note) and a
    `_slide` version without them, because the slide itself carries the headline and source."""
    headline = fig.suptitle(title, x=0.01, ha="left", fontsize=14, fontweight="bold", color=INK, wrap=True)
    source = fig.text(0.01, 0.01, note, fontsize=note_size, color=MUTED, wrap=True)
    fig.tight_layout(rect=(0, bottom, 1, top))
    fig.savefig(EXHIBITS / filename, dpi=200)
    plt.show()
    headline.set_visible(False)
    source.set_visible(False)
```

```
    fig.tight_layout(rect=(0, 0, 1, 1))
    fig.savefig(EXHIBITS / filename.replace(".png", "_slide.png"), dpi=200)


def hide_spines(ax, sides=("top", "right")):
    for side in sides:
        ax.spines[side].set_visible(False)


print("Setup done. Charts will be saved to", EXHIBITS.resolve())
```

```
Setup done. Charts will be saved to /Users/kaichung/Documents/BUSN7900/Business_Case/data/exhibits
```

1. Trade data: Eurostat Comext (feeds Slides 2 and 3)

Source. Eurostat, Comext database, dataset **DS-045409** ("EU trade since 1988 by HS2-4-6 and CN8"). Guide: https://ec.europa.eu/eurostat/en/web/user-guides/data-browser/api-data-access/api-getting-started/comext-database

Download settings used (Eurostat data browser, export as CSV in the "linear" layout, saved as `data/raw/comext/Comext_final.csv`):

- Frequency monthly, January 2019 to August 2026
- Reporters: the EU member states (26 have rows in this file; Luxembourg has none) plus the `EU27_2020` total
- Partner: China (`CN`), which for imports means country of origin
- Products: CN8 codes `87038010` and `87038090` (battery-electric passenger cars, new and other)
- Flows: `1` imports and `2` exports (only imports are used)
- Indicators: `VALUE_IN_EUROS` , `SUPPLEMENTARY_QUANTITY` (number of cars) and `QUANTITY_IN_100KG`

What the code does.

1. Keeps imports from the member states and drops the `EU27_2020` row, because it is only the sum of the members and would double count.
2. Lays out one row per country, month and product, with value, cars and weight in separate columns.
3. Flags suspect rows with the rule in section 0 and keeps both a raw and a cleaned EU total.
4. Checks that the member total equals Eurostat's own EU total, then saves two tables to `data/processed/` .

Limits to state in the talk.

- The value is the customs value (roughly cost, insurance and freight). It excludes duty, VAT, shipping to the dealer and dealer margin, so it is not a shelf price.
- It covers every maker that exports cars from China, including Western brands, so it is not BYD alone and the mix of models matters.
- Countries report where the car clears customs (Belgium is large because of its ports), so use EU totals, not country splits.
- 2019 and 2020 volumes are tiny, so summaries start in 2021.

In [2]:
```
# Find the Eurostat file: the data folder first, then next to the notebook.
candidates = [RAW_COMEXT / "Comext_final.csv", Path("Comext_final.csv")]
PATH = next((p for p in candidates if p.exists()), None)
if PATH is None:
    raise FileNotFoundError("Put Comext_final.csv in data/raw/comext/ (see the download settings above).")
print("Reading", PATH)

raw = pd.read_csv(PATH, dtype=str)
raw["value"] = pd.to_numeric(raw["OBS_VALUE"], errors="coerce")
raw["month"] = pd.to_datetime(raw["TIME_PERIOD"], format="%Y-%m")
print(f"Months in file: {raw['month'].min():%Y-%m} to {raw['month'].max():%Y-%m}")

# 1) imports only, member countries only (EU27_2020 is just the sum of the members)
imp = raw[(raw["flow"] == "1") & (raw["reporter"] != "EU27_2020")]

# 2) one row per reporter, month and product, one column per indicator
t = (imp.pivot_table(index=["month", "reporter", "product"], columns="indicators", values="value", aggfunc="sum")
        .reset_index()
        .rename(columns={"VALUE_IN_EUROS": "value_eur", "SUPPLEMENTARY_QUANTITY": "units",
                         "QUANTITY_IN_100KG": "weight_100kg"}))
t["eur_per_car"] = t["value_eur"] / t["units"]
t["kg_per_car"] = t["weight_100kg"] * 100 / t["units"]

# 3) flag rows where value or weight per car is implausible for a car (likely reporting errors)
t["suspect"] = ((t["eur_per_car"] < MIN_EUR_PER_CAR) | (t["kg_per_car"] < MIN_KG_PER_CAR)) & (t["units"] >= MIN_UNI
print(f"{t['suspect'].sum()} of {len(t)} reporter-month rows flagged as suspect")

# 4) EU monthly totals: raw (all rows) and cleaned (suspect rows removed)
def monthly(d):
```

```
    m = d.groupby("month")[["value_eur", "units"]].sum()
    m["eur_per_car"] = m["value_eur"] / m["units"]
    return m

eu = monthly(t).join(monthly(t[~t["suspect"]]), rsuffix="_clean")
eu["year"] = eu.index.year

# Check: our member total must equal Eurostat's own EU27_2020 total, month by month
eu_total = (raw[(raw["flow"] == "1") & (raw["reporter"] == "EU27_2020") & (raw["indicators"] == "SUPPLEMENTARY_QUAN
            .groupby("month")["value"].sum())
member_total = imp[imp["indicators"] == "SUPPLEMENTARY_QUANTITY"].groupby("month")["value"].sum()
largest_gap = (eu_total - member_total).abs().max()
assert largest_gap < 1, f"Member total differs from the EU27 total by {largest_gap:,.0f} cars"
print(f"Check passed: member states add up to the EU27 total (largest monthly gap {largest_gap:,.0f} cars)")

# 5) annual summary, 2021 onward (2026 covers only the months available)
a = eu[eu["year"] >= 2021].groupby("year")[["value_eur", "units", "value_eur_clean", "units_clean"]].sum()
a["eur_per_car_raw"] = a["value_eur"] / a["units"]
a["eur_per_car_clean"] = a["value_eur_clean"] / a["units_clean"]
display(a.round(0))

t.to_csv(PROCESSED / "trade_by_reporter_month.csv", index=False)
eu.to_csv(PROCESSED / "trade_eu_monthly.csv")
```

```
Reading Comext_final.csv
Months in file: 2019-01 to 2026-08
350 of 2394 reporter-month rows flagged as suspect
Check passed: member states add up to the EU27 total (largest monthly gap 0 cars)
```

indicators	value_eur	units	value_eur_clean	units_clean	eur_per_car_raw	eur_per_car_clean
year						
2021	3.499114e+09	209029.0	3.468931e+09	177335.0	16740.0	19561.0
2022	8.137178e+09	344060.0	8.123454e+09	329425.0	23650.0	24659.0
2023	1.103492e+10	491502.0	1.101079e+10	470109.0	22451.0	23422.0
2024	8.883807e+09	439822.0	8.846867e+09	414343.0	20199.0	21352.0
2025	6.350615e+09	431061.0	6.269150e+09	355183.0	14733.0	17650.0
2026	6.064056e+09	464216.0	6.036956e+09	387159.0	13063.0	15593.0

2. Inputs entered by hand

These come from company pages and press releases, so they are typed in with a source and a retrieval date. Each cell saves a small CSV in `data/inputs/` that the charts read.

2.1 Entry-model list prices (Slide 3)

Cheapest battery-electric model of each maker on its German website, list price including 19% VAT, read on 27 Oct 2026. The BYD configurator also showed a promotional price of EUR 14,190 against the EUR 23,490 list price; the list price is used because a promotion is temporary. Check before presenting whether the VW price is before or after any purchase subsidy.

In [3]:
```
prices_in = pd.DataFrame([
    {"maker": "VW", "model": "ID. Polo Trend", "list_price_eur": 24995, "vat_rate": DEFAULT_VAT,
     "price_date": "2026-10-27",
     "source": "https://www.volkswagen.de/de/modelle.html/__app/id-polo.app",
     "note": "Entry variant, list price incl. VAT; no purchase-subsidy badge on the Trend variant"},
    {"maker": "BYD", "model": "Dolphin Surf Active", "list_price_eur": 23490, "vat_rate": DEFAULT_VAT,
     "price_date": "2026-10-27",
     "source": "https://www.byd.com/de/konfiguration/dolphin-surf",
     "note": "List price incl. VAT; configurator also showed a promotional price of 14,190, not used"},
])
prices_in.to_csv(INPUTS / "entry_prices.csv", index=False)
display(prices_in)
```

	maker	model	list_price_eur	vat_rate	price_date	source	note
0	VW	ID. Polo Trend	24995	0.19	2026-10-27	https://www.volkswagen.de/de/modelle.html/__ap...	Entry variant, list price incl. VAT; no purcha...
1	BYD	Dolphin Surf Active	23490	0.19	2026-10-27	https://www.byd.com/de/konfiguration/dolphin-surf	List price incl. VAT; configurator also showed...

2.2 VW operating margins and the margin target (Slide 1)

Operating return on sales. Sources: VW Group press release no. 69/2026 (24 Jul 2026), Brand Group Core press release no. 68/2026 (24 Jul 2026), and the VW newsroom page for the 6.5% target. Full-year 2024 and 2025 margins are calculated from the rounded operating result and revenue stated in the release, so confirm them against the annual report before quoting them. The 6.5% target applies to the VW Passenger Cars brand, not the Group, and was set in 2023 for 2026.

In [4]:
```
SRC_GROUP = "VW Group press release no. 69/2026 (24 Jul 2026)"
SRC_CORE = "VW press release no. 68/2026, Brand Group Core (24 Jul 2026)"
SRC_TARGET = "VW newsroom, 'ACCELERATE FORWARD | Road to 6.5' (2023); confirm not revised"
financials = pd.DataFrame([
    ("FY2024", "Group", 5.9, "Computed: 19.1 / 324.7 (release, About section)"),
    ("FY2025", "Group", 2.8, "Computed: 8.9 / 321.9 (release, About section); verify vs annual report"),
    ("H1 2025", "Group", 4.2, SRC_GROUP),
    ("H1 2026", "Group", 3.8, SRC_GROUP),
    ("H1 2026", "Group before special items", 4.3, SRC_GROUP),
    ("H1 2025", "Core brand group", 4.8, SRC_GROUP),
    ("H1 2026", "Core brand group", 4.9, SRC_GROUP),
    ("H1 2026", "Progressive brand group", 3.8, SRC_GROUP),
    ("H1 2026", "Sport Luxury brand group", 8.0, SRC_GROUP),
    ("H1 2026", "VW Passenger Cars", 2.4, SRC_CORE),
    ("H1 2025", "VW Passenger Cars", 2.5, SRC_CORE),
    ("H1 2026", "VW Passenger Cars before special items", 3.8, SRC_CORE),
    ("H1 2026", "Skoda Auto", 8.5, SRC_CORE),
    ("H1 2026", "SEAT/CUPRA", 1.6, SRC_CORE),
    ("H1 2026", "VW Commercial Vehicles", 3.3, SRC_CORE),
    ("H1 2026", "Core brand group before special items", 5.9, SRC_CORE),
    ("2026 target", "VW Passenger Cars", 6.5, SRC_TARGET),
], columns=["period", "entity", "operating_margin_pct", "source"])
financials["retrieved"] = "2026-10-27"
financials.to_csv(INPUTS / "vw_financials.csv", index=False)
display(financials)
```

	period	entity	operating_margin_pct	source	retrieved
0	FY2024	Group	5.9	Computed: 19.1 / 324.7 (release, About section)	2026-10-27
1	FY2025	Group	2.8	Computed: 8.9 / 321.9 (release, About section)...	2026-10-27
2	H1 2025	Group	4.2	VW Group press release no. 69/2026 (24 Jul 2026)	2026-10-27
3	H1 2026	Group	3.8	VW Group press release no. 69/2026 (24 Jul 2026)	2026-10-27
4	H1 2026	Group before special items	4.3	VW Group press release no. 69/2026 (24 Jul 2026)	2026-10-27
5	H1 2025	Core brand group	4.8	VW Group press release no. 69/2026 (24 Jul 2026)	2026-10-27
6	H1 2026	Core brand group	4.9	VW Group press release no. 69/2026 (24 Jul 2026)	2026-10-27
7	H1 2026	Progressive brand group	3.8	VW Group press release no. 69/2026 (24 Jul 2026)	2026-10-27
8	H1 2026	Sport Luxury brand group	8.0	VW Group press release no. 69/2026 (24 Jul 2026)	2026-10-27
9	H1 2026	VW Passenger Cars	2.4	VW press release no. 68/2026, Brand Group Core...	2026-10-27
10	H1 2025	VW Passenger Cars	2.5	VW press release no. 68/2026, Brand Group Core...	2026-10-27
11	H1 2026	VW Passenger Cars before special items	3.8	VW press release no. 68/2026, Brand Group Core...	2026-10-27
12	H1 2026	Skoda Auto	8.5	VW press release no. 68/2026, Brand Group Core...	2026-10-27
13	H1 2026	SEAT/CUPRA	1.6	VW press release no. 68/2026, Brand Group Core...	2026-10-27
14	H1 2026	VW Commercial Vehicles	3.3	VW press release no. 68/2026, Brand Group Core...	2026-10-27
15	H1 2026	Core brand group before special items	5.9	VW press release no. 68/2026, Brand Group Core...	2026-10-27
16	2026 target	VW Passenger Cars	6.5	VW newsroom, 'ACCELERATE FORWARD | Road to 6.5...	2026-10-27

2.3 Reported cost savings (Slide 6)

Only published or reported figures, each with its basis. The 2024 and 2025 savings come from a press report (electrive, 17 Feb 2026), not from a VW release, so replace them with VW's own figure if one is found. The EUR 1.5B labour figure is the annual saving targeted by 2030, quoted in a press report (electrive, 18 May 2026); look for VW's own statement. The bases differ (achieved per year, targeted per year by 2030, cumulative), so the figures must not be added together.

In [5]:
```
savings = pd.DataFrame([
    {"group": "Already delivered (reported)", "item": "Savings in 2024", "eur_bn": 15.0, "basis": "per year",
     "source": "electrive, 17 Feb 2026 (press report, not a VW release)"},
    {"group": "Already delivered (reported)", "item": "Savings in 2025 (expected)", "eur_bn": 18.0, "basis": "per y
     "source": "electrive, 17 Feb 2026 (press report, not a VW release)"},
    {"group": "Newly announced", "item": "Labour costs", "eur_bn": 1.5, "basis": "per year by 2030",
     "source": "electrive, 18 May 2026 (press report quoting VW: annual labour cost savings of EUR 1.5B by 2030)"},
    {"group": "Newly announced", "item": "Production steering, Core group", "eur_bn": 1.0, "basis": "cumulative to
     "source": "VW press release no. 68/2026 (24 Jul 2026)"},
])
savings.to_csv(INPUTS / "savings_reported.csv", index=False)
display(savings)
```

	group	item	eur_bn	basis	source
0	Already delivered (reported)	Savings in 2024	15.0	per year	electrive, 17 Feb 2026 (press report, not a VW...
1	Already delivered (reported)	Savings in 2025 (expected)	18.0	per year	electrive, 17 Feb 2026 (press report, not a VW...
2	Newly announced	Labour costs	1.5	per year by 2030	electrive, 18 May 2026 (press report quoting V...
3	Newly announced	Production steering, Core group	1.0	cumulative to 2030	VW press release no. 68/2026 (24 Jul 2026)

3. Exhibits

Slide order (1, 2, 3, 6). Each chart reads only the saved tables from sections 1 and 2.

3.1 Slide 1: VW's own brand is far below its margin target

Message. VW Passenger Cars earned 2.4% in the first half of 2026 against its 6.5% target, while Skoda, in the same group, earned 8.5%. The hollow marker shows the 3.8% margin before special items (the halt of ID.4 production in the US and restructuring), so the gap does not rest on one-off costs alone.

Limits. Half-year figures; full-year results can differ. The chart shows that the gap exists, not what caused it: VW's own release also cites price promotions, mix and a weak market.

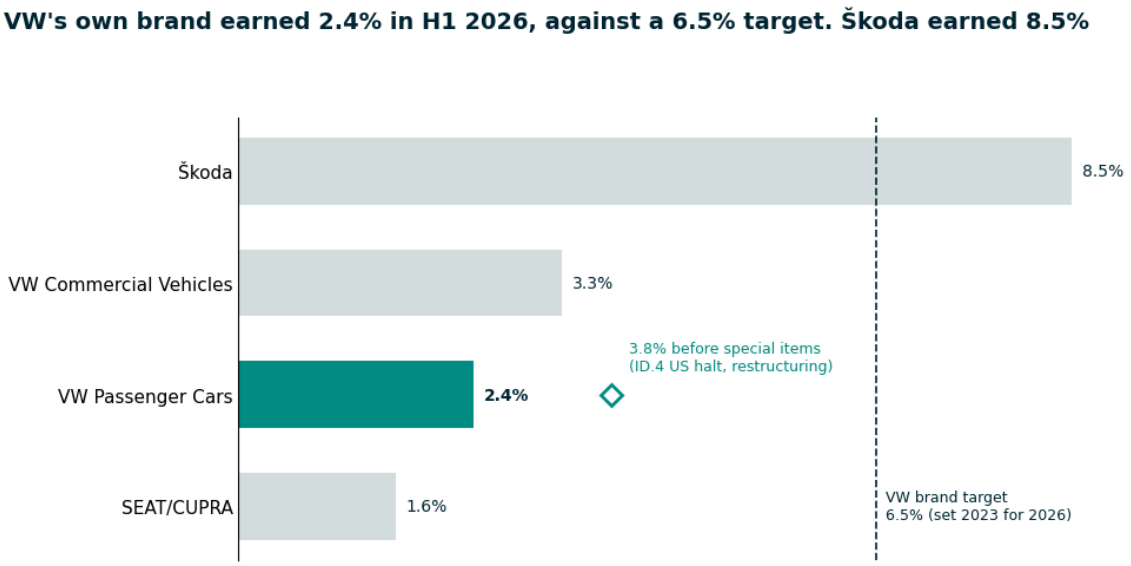
In [6]:
```
fin = pd.read_csv(INPUTS / "vw_financials.csv")

def margin_of(period, entity):
    return float(fin[(fin["period"] == period) & (fin["entity"] == entity)]["operating_margin_pct"].iloc[0])

brands = {"Škoda": margin_of("H1 2026", "Skoda Auto"),
          "VW Commercial Vehicles": margin_of("H1 2026", "VW Commercial Vehicles"),
          "VW Passenger Cars": margin_of("H1 2026", "VW Passenger Cars"),
          "SEAT/CUPRA": margin_of("H1 2026", "SEAT/CUPRA")}
vw = brands["VW Passenger Cars"]
vw_adj = margin_of("H1 2026", "VW Passenger Cars before special items")
target = margin_of("2026 target", "VW Passenger Cars")
skoda = brands["Škoda"]
order = sorted(brands, key=brands.get)                    # lowest at the bottom, highest at the top

title = (f"VW's own brand earned {vw:.1f}% in H1 2026, against a {target:.1f}% target. "
         f"Škoda earned {skoda:.1f}%")

fig, ax = plt.subplots(figsize=(11, 5.2))
ax.barh(order, [brands[b] for b in order], height=0.6,
        color=[ACCENT if b == "VW Passenger Cars" else GREY for b in order])
for i, b in enumerate(order):
    ax.text(brands[b] + 0.1, i, f"{brands[b]:.1f}%", va="center", color=INK,
            fontweight="bold" if b == "VW Passenger Cars" else "normal")
i_vw = order.index("VW Passenger Cars")
ax.plot(vw_adj, i_vw, marker="D", mfc="white", mec=ACCENT, mew=2, ms=9, ls="none", zorder=5)
ax.text(vw_adj + 0.18, i_vw + 0.33, f"{vw_adj:.1f}% before special items\n(ID.4 US halt, restructuring)",
        color=ACCENT, fontsize=9, va="center")
ax.axvline(target, color=INK, ls="--", lw=1.2)
ax.text(target + 0.1, 0, f"VW brand target\n{target:.1f}% (set 2023 for 2026)", color=INK, fontsize=9, va="center")
ax.set_xlim(0, 10.5)
ax.set_xticks([])
ax.tick_params(axis="y", length=0, labelsize=11)
hide_spines(ax, ("top", "right", "bottom"))
finish(fig, title,
       "Operating return on sales, H1 2026, as reported (incl. special items). Half-year figures; full-year results
       "Source: Volkswagen press release no. 68/2026 (24 Jul 2026); target: VW newsroom, 'Road to 6.5' (2023).",
       "01_margin_gap.png", top=0.90)
print(f"VW Passenger Cars: {vw:.1f}% reported, {vw_adj:.1f}% before special items, target {target:.1f}%")
print(f"Gap to target: {target - vw:.1f} points reported, {target - vw_adj:.1f} points before special items")
print(f"Škoda is {skoda / vw:.1f}x VW Passenger Cars' margin")
```




```
VW Passenger Cars: 2.4% reported, 3.8% before special items, target 6.5%
Gap to target: 4.1 points reported, 2.7 points before special items
Škoda is 3.5x VW Passenger Cars' margin
```

3.2 Slide 2: imports dipped after the duties, then rose above their old level

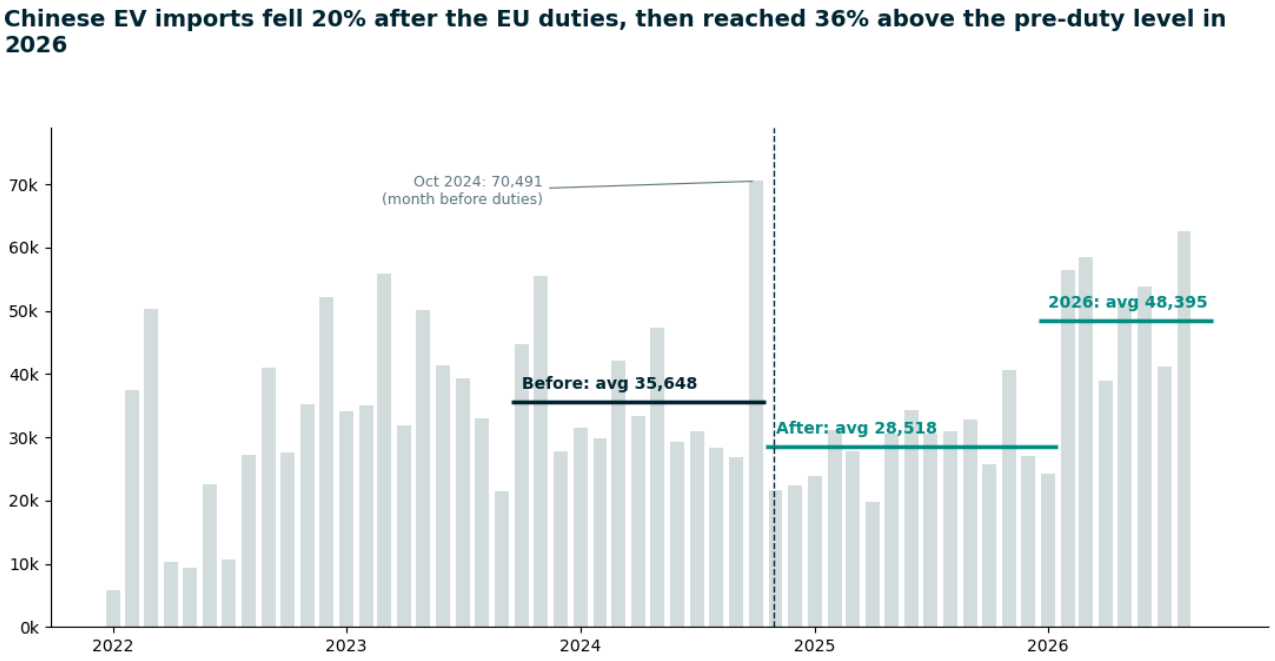
Message. Cars imported from China per month fell after the duties began on 30 October 2024 and then recovered to a level above the 12 months before. Three average lines compare the periods; October 2024 itself (the month before the duties took effect) is shown but left out of the averages.

Limits. Cleaned data (suspect rows removed). 2026 covers January to the latest month in the file only. The chart cannot separate BYD from other makers, and the rise in 2026 is consistent with, but does not prove, a response to weak demand in China.

In [7]:
```
eu = pd.read_csv(PROCESSED / "trade_eu_monthly.csv", parse_dates=["month"]).set_index("month")
cars = eu["units_clean"]                                  # cleaned monthly car counts

periods = {"Before": PERIOD_BEFORE, "After": PERIOD_AFTER,
           "2026": (PERIOD_LATEST_START, str(eu.index.max().date()))}
units_avg = {k: cars[a:b].mean() for k, (a, b) in periods.items()}
base = units_avg["Before"]
d_after = units_avg["After"] / base - 1
d_latest = units_avg["2026"] / base - 1
title = (f"Chinese EV imports {'fell' if d_after < 0 else 'rose'} {abs(d_after):.0%} after the EU duties, "
         f"then reached {abs(d_latest):.0%} {'above' if d_latest > 0 else 'below'} the pre-duty level in 2026")

d = cars[cars.index >= "2022-01-01"]
fig, ax = plt.subplots(figsize=(11, 6))
ax.bar(d.index, d.values, width=22, color=GREY)
for name, (a, b) in periods.items():                      # average line per period
    colour = INK if name == "Before" else ACCENT
    ax.hlines(units_avg[name], pd.Timestamp(a) - pd.Timedelta(days=15), pd.Timestamp(b) + pd.Timedelta(days=45),
              color=colour, lw=2.5)
    ax.text(pd.Timestamp(a), units_avg[name] + 1500, f"{name}: avg {units_avg[name]:,.0f}", color=colour,
            fontsize=10, fontweight="bold", va="bottom")
ax.axvline(DUTY_DATE, color=INK, ls="--", lw=1)
ax.annotate(f"Oct 2024: {cars['2024-10-01']:,.0f}\n(month before duties)", (pd.Timestamp("2024-10-01"), cars["2024-
            xytext=(-130, -6), textcoords="offset points", fontsize=9, color=MUTED, ha="right", va="center",
            arrowprops=dict(arrowstyle="-", color=MUTED, lw=0.8))
ax.set_ylim(0, cars.max() * 1.12)
ax.yaxis.set_major_formatter(lambda v, _: f"{v/1000:,.0f}k")
hide_spines(ax)
finish(fig, title,
       "Grey bars = cars imported per month from China (CN 8703 80), suspect reporting rows removed. "
       "Dashed line = duties start 30 Oct 2024. Source: Eurostat Comext.",
       "02_import_trend.png", bottom=0.04, note_size=8.5)
for k, v in units_avg.items():
    print(f"{k}: {v:,.0f} cars a month ({v / base - 1:+.0%} vs before)")
```




```
Before: 35,648 cars a month (+0% vs before)
After: 28,518 cars a month (−20% vs before)
2026: 48,395 cars a month (+36% vs before)
```

3.3 Slide 3: the average Chinese import still lands near VW's entry price

Message. The average customs value per car fell after the duties. Adding the 10% standard and 17% BYD duties (dashed line, illustrative) gives about EUR 19,700, level with BYD's cheapest model and below VW's cheapest, once VAT is taken off both list prices.

Limits. This is a floor, not a shelf price: the customs value excludes shipping, dealer margin and the maker's own markup, and the 27% rate is BYD's, not every maker's (other rates: Tesla 7.8% + 10%, Geely 18.8% + 10%, SAIC 35.3% + 10%). Say "lands at about VW's entry price", not "undercuts VW".

In [8]:
```
prices = pd.read_csv(INPUTS / "entry_prices.csv")
prices["vat_rate"] = prices["vat_rate"].fillna(DEFAULT_VAT)
prices["ex_vat"] = prices["list_price_eur"] / (1 + prices["vat_rate"])      # remove VAT to compare with customs va

# Average customs value per car, smoothed over 3 months (total value / total cars, not an average of averages)
v3 = eu["value_eur_clean"].rolling(3).sum() / eu["units_clean"].rolling(3).sum()
v3_duty = v3 * (1 + STANDARD_DUTY + BYD_DUTY)           # illustrative: the same cars with BYD-rate duties added

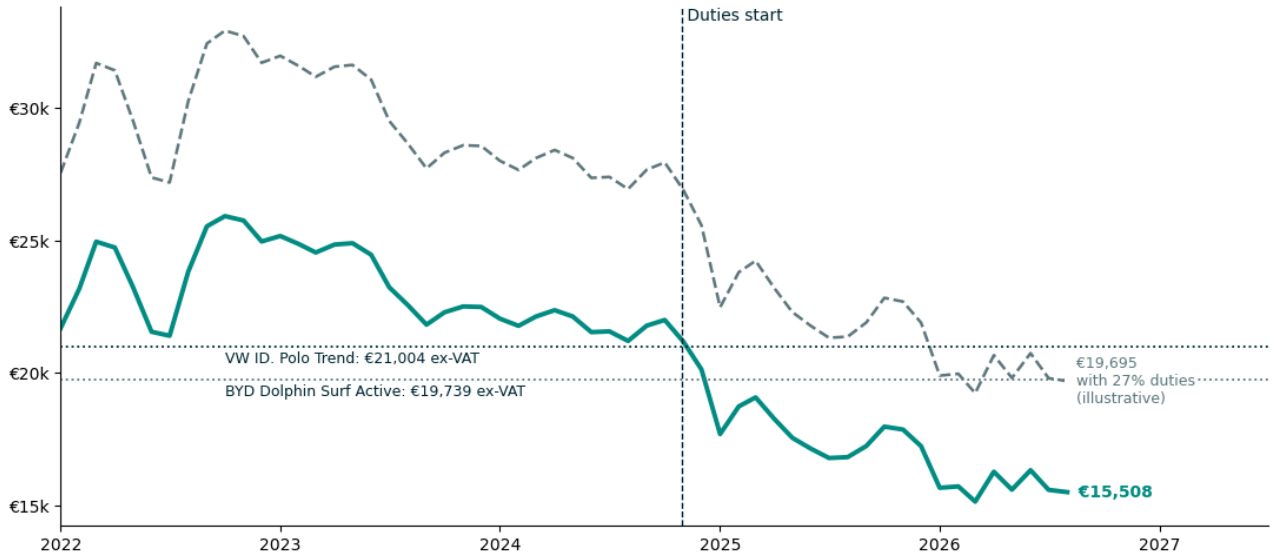
def value_per_car(a, b):
    s = eu.loc[a:b]
    return s["value_eur_clean"].sum() / s["units_clean"].sum()

value_avg = {k: value_per_car(*v) for k, v in periods.items()}
chg = value_avg["2026"] / value_avg["Before"] - 1
title = (f"The average customs value of Chinese EVs {'fell' if chg < 0 else 'rose'} {abs(chg):.0%} after the duties
         f"to €{value_avg['2026']:,.0f} per car in 2026")

d = v3[v3.index >= "2022-01-01"].dropna()
last = d.index[-1]
fig, ax = plt.subplots(figsize=(11, 6))
ax.plot(d.index, d.values, color=ACCENT, lw=2.8)
ax.plot(d.index, v3_duty[d.index].values, color=MUTED, lw=1.8, ls="--")
ax.text(last, d.values[-1], f"  €{d.values[-1]:,.0f}", color=ACCENT, fontweight="bold", va="center")
ax.text(last, v3_duty[last], f"  €{v3_duty[last]:,.0f}\n  with 27% duties\n  (illustrative)",
        color=MUTED, fontsize=9, va="center", bbox=dict(fc="white", ec="none", pad=1.5), zorder=5)
for _, r in prices.dropna(subset=["list_price_eur"]).iterrows():
    ax.axhline(r["ex_vat"], color=INK if str(r["maker"]).upper() == "VW" else MUTED, ls=":", lw=1.3)
    ax.text(pd.Timestamp("2022-10-01"), r["ex_vat"] - 150, f"{r['maker']} {r['model']}: €{r['ex_vat']:,.0f} ex-VAT"
            fontsize=9, color=INK, va="top", bbox=dict(fc="white", ec="none", pad=1))
ax.axvline(DUTY_DATE, color=INK, ls="--", lw=1)
ax.text(DUTY_DATE, ax.get_ylim()[1], " Duties start", va="top", color=INK, fontsize=10)
ax.set_xlim(d.index[0], last + pd.DateOffset(months=11))
ax.yaxis.set_major_locator(MultipleLocator(5000))              # ticks every EUR 5,000 so labels are exact
ax.yaxis.set_major_formatter(lambda v, _: f"€{v/1000:,.0f}k")
hide_spines(ax)
```

```
finish(fig, title,
       "Teal line = average customs value per car imported from China, 3-month average, suspect rows removed (all m
       "models, so mix matters). Dashed grey = same value plus 10% + 17% duties; excludes VAT, shipping and dealer
       "Source: Eurostat Comext.",
       "03_price_trend.png")
for k, v in value_avg.items():
    print(f"{k}: €{v:,.0f} average customs value per car")
print(f"Latest 3-month: €{d.values[-1]:,.0f} | with 27% duties: €{v3_duty[last]:,.0f}")
for _, r in prices.dropna(subset=["list_price_eur"]).iterrows():
    gap = v3_duty[last] - r["ex_vat"]
    print(f"{r['maker']} {r['model']}: €{r['ex_vat']:,.0f} ex-VAT | duty-inclusive Chinese average is "
          f"€{abs(gap):,.0f} {'below' if gap < 0 else 'above'} this price")
```

The average customs value of Chinese EVs fell 29% after the duties, to €15,593 per car in 2026



Teal line = average customs value per car imported from China, 3-month average, suspect rows removed (all makers and models, so mix matters). Dashed grey = same value plus 10% + 17% duties; excludes VAT, shipping and dealer margin. Source: Eurostat Comext.

```
Before: €22,018 average customs value per car
After: €17,531 average customs value per car
2026: €15,593 average customs value per car
Latest 3-month: €15,508 | with 27% duties: €19,695
VW ID. Polo Trend: €21,004 ex-VAT | duty-inclusive Chinese average is €1,309 below this price
BYD Dolphin Surf Active: €19,739 ex-VAT | duty-inclusive Chinese average is €44 below this price
```

3.4 Slide 6: large savings are already reported, the margin is still thin

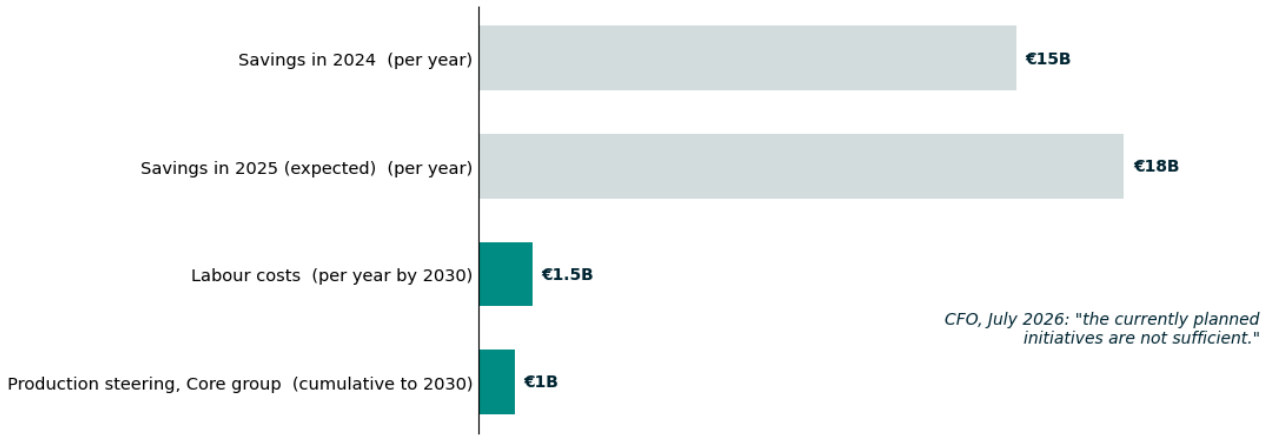
Message. VW reports yearly savings of about EUR 15B to 18B, yet the Group margin was 3.8% in the first half of 2026. The newly announced production and labour savings are small beside that, and VW's CFO said in July 2026 that "the currently planned initiatives are not sufficient."

Limits. The bars have different bases (per year, by 2030, cumulative) and are not additive. Savings offset inflation and other headwinds, so they do not convert directly into margin points. All figures except the production one come from press reports.

In [9]:
```
sav = pd.read_csv(INPUTS / "savings_reported.csv")
margin = float(fin[(fin["period"] == "H1 2026") & (fin["entity"] == "Group")]["operating_margin_pct"].iloc[0])
delivered = sav[sav["group"].str.startswith("Already")]["eur_bn"].max()
title = f"VW reports about €{delivered:.0f}B in yearly savings, yet its H1 2026 margin is {margin:.1f}%"

rows = sav.iloc[::-1].reset_index(drop=True)            # first row at the top
fig, ax = plt.subplots(figsize=(11, 5.2))
ax.barh([f"{i}  ({b})" for i, b in zip(rows["item"], rows["basis"])], rows["eur_bn"], height=0.6,
        color=[GREY if g.startswith("Already") else ACCENT for g in rows["group"]])
for i, v in enumerate(rows["eur_bn"]):
    ax.text(v + 0.25, i, f"€{v:g}B", va="center", fontweight="bold", color=INK)
ax.set_xlim(0, 22)
ax.set_xticks([])
ax.tick_params(axis="y", length=0, labelsize=10.5)
hide_spines(ax, ("top", "right", "bottom"))
ax.text(21.8, 0.5, "CFO, July 2026: \"the currently planned\ninitiatives are not sufficient.\"",
        ha="right", va="center", fontsize=10, color=INK, style="italic")
finish(fig, title,
       "Grey = already delivered; teal = newly announced. Bases differ (per year vs cumulative), so bars are not ad
```

```
      "Savings offset inflation and other headwinds, so they do not convert directly into margin points. "
      "Sources: electrive (17 Feb and 18 May 2026); VW press releases no. 68 and 69/2026.",
      "06_savings_vs_margin.png", bottom=0.06, top=0.88)
display(sav)
```



	group	item	eur_bn	basis	source
0	Already delivered (reported)	Savings in 2024	15.0	per year	electrive, 17 Feb 2026 (press report, not a VW...
1	Already delivered (reported)	Savings in 2025 (expected)	18.0	per year	electrive, 17 Feb 2026 (press report, not a VW...
2	Newly announced	Labour costs	1.5	per year by 2030	electrive, 18 May 2026 (press report quoting V...
3	Newly announced	Production steering, Core group	1.0	cumulative to 2030	VW press release no. 68/2026 (24 Jul 2026)

3.5 Slide 7: size the four cost levers (EUR per car)

VW does not publish EUR-per-car savings for any lever (they sit in internal cost models and supplier contracts), so each figure here is a **team estimate** built from published inputs plus clearly labelled assumptions. Every input row has a `type`:

- `sourced`: a published figure, with its page or link in `source`;
- `assumption`: the team's judgement, entered as a low and a high value; change these to test the result;
- `to source`: still empty; the lever shows `NaN` and stays off the slide until you fill it in (none left at present).

Lever	Method
Shared platform	Group R&D ratio × entry-car price ex-VAT = R&D embedded in one car; × platform share of R&D × cut from sharing across four models
External software	CARIAD 2025 operating loss ÷ Group deliveries = software cost per car; × share avoided with an external partner
Battery chemistry	pack size (kWh) × pack cost today (USD/kWh, BNEF) × cost reduction from chemistry / unified cell ÷ USD per EUR
Sourcing	entry-car price ex-VAT (cost proxy) × UBS cost gap to BYD × share of the gap re-tendering can close

The levers overlap (a cheaper battery changes platform cost), so **do not add them together**. Low case = smallest saving; high case = largest.

In [10]:
```
# Fill in or change low / high; keep `source` and `type` honest. Re-run this cell and the ones below.
vw_price = float(prices.loc[prices["maker"].str.contains("VW|Volkswagen", case=False), "ex_vat"].iloc[0])   # Slide
AR25 = "VW Annual Report 2025 (PDF page)"
lever_inputs = pd.DataFrame([
    # lever,               input,                                        low,        high,       unit,   type,
    ("Shared platform",   "R&D ratio, Automotive Division 2025",          6.7,        6.7,        "%",      "sourced
    ("Shared platform",   "entry car price ex-VAT",                       vw_price,   vw_price,  "EUR",    "sourced
    ("Shared platform",   "platform share of R&D",                        30.0,       50.0,       "%",      "assumpt
```

```
    ("Shared platform",   "cut from sharing across four models",          20.0,       40.0,      "%",      "assumpt
    ("External software", "CARIAD operating loss 2025",                   2180.0,     2180.0,    "EUR m",  "sourced
    ("External software", "Group deliveries 2025",                        8983900.0,  8983900.0, "cars",   "sourced
    ("External software", "share of software cost avoided with partner",  20.0,       50.0,      "%",      "assumpt
    ("Battery chemistry", "battery pack size",                            37.0,       37.0,      "kWh",    "sourced
    ("Battery chemistry", "pack cost today",                              99.0,       131.0,     "USD/kWh","sourced
    ("Battery chemistry", "cost reduction from chemistry / unified cell", 10.0,       25.0,      "%",      "assumpt
    ("Battery chemistry", "exchange rate",                                1.10,       1.20,      "USD per EUR", "as
    ("Sourcing",          "entry car price ex-VAT (cost proxy)",          vw_price,   vw_price,  "EUR",    "sourced
    ("Sourcing",          "cost gap to BYD Seal",                         25.0,       35.0,      "%",      "sourced
    ("Sourcing",          "share of the gap closable by re-tendering",    5.0,        15.0,      "%",      "assumpt
], columns=["lever", "input", "low", "high", "unit", "type", "source"])
lever_inputs[["low", "high"]] = lever_inputs[["low", "high"]].astype(float)
lever_inputs.to_csv(INPUTS / "lever_inputs.csv", index=False)
display(lever_inputs)
```

	lever	input	low	high	unit	type	source
0	Shared platform	R&D ratio, Automotive Division 2025	6.700000e+00	6.700000e+00	%	sourced	VW Annual Report 2025 (PDF page) p119
1	Shared platform	entry car price ex-VAT	2.100420e+04	2.100420e+04	EUR	sourced	VW ID. Polo Trend list price, Slide 3
2	Shared platform	platform share of R&D	3.000000e+01	5.000000e+01	%	assumption	Team judgement: no VW disclosure
3	Shared platform	cut from sharing across four models	2.000000e+01	4.000000e+01	%	assumption	Team judgement; four models on MEB+ (Annual Re...
4	External software	CARIAD operating loss 2025	2.180000e+03	2.180000e+03	EUR m	sourced	VW Annual Report 2025 (PDF page) p458, p460
5	External software	Group deliveries 2025	8.983900e+06	8.983900e+06	cars	sourced	VW Annual Report 2025 (PDF page) p97
6	External software	share of software cost avoided with partner	2.000000e+01	5.000000e+01	%	assumption	Team judgement; Rivian zonal architecture from...
7	Battery chemistry	battery pack size	3.700000e+01	3.700000e+01	kWh	sourced	electrive, 15 Dec 2025 (ID. Polo small LFP pack)
8	Battery chemistry	pack cost today	9.900000e+01	1.310000e+02	USD/kWh	sourced	BNEF Dec 2025 via ess-news.com: EV pack averag...
9	Battery chemistry	cost reduction from chemistry / unified cell	1.000000e+01	2.500000e+01	%	assumption	Team judgement; VW's own aim is up to 50% in t...
10	Battery chemistry	exchange rate	1.100000e+00	1.200000e+00	USD per EUR	assumption	Check against the ECB reference rate and narrow
11	Sourcing	entry car price ex-VAT (cost proxy)	2.100420e+04	2.100420e+04	EUR	sourced	VW ID. Polo Trend list price, Slide 3; price o...
12	Sourcing	cost gap to BYD Seal	2.500000e+01	3.500000e+01	%	sourced	UBS teardown (2023) via TechNode, 6 Sep 2023: ...
13	Sourcing	share of the gap closable by re-tendering	5.000000e+00	1.500000e+01	%	assumption	Team judgement: UBS credits BYD's ~75% in-hous...

In [11]:

```
def get(lever, name):
    r = lever_inputs[(lever_inputs["lever"] == lever) & (lever_inputs["input"].str.startswith(name))].iloc[0]
    return float(r["low"]), float(r["high"])

rd_lo, rd_hi = get("Shared platform", "R&D ratio")
pr_lo, pr_hi = get("Shared platform", "entry car price")
ps_lo, ps_hi = get("Shared platform", "platform share")
cu_lo, cu_hi = get("Shared platform", "cut from sharing")
loss_lo, loss_hi = get("External software", "CARIAD")
dl_lo, dl_hi = get("External software", "Group deliveries")
av_lo, av_hi = get("External software", "share of software")
kwh_lo, kwh_hi = get("Battery chemistry", "battery pack")
pk_lo, pk_hi = get("Battery chemistry", "pack cost today")
br_lo, br_hi = get("Battery chemistry", "cost reduction")
fx_lo, fx_hi = get("Battery chemistry", "exchange rate")
sp_lo, sp_hi = get("Sourcing", "entry car price")
ug_lo, ug_hi = get("Sourcing", "cost gap")
sc_lo, sc_hi = get("Sourcing", "share of the gap")

rd_per_car = rd_lo / 100 * pr_lo                                  # R&D embedded in one entry car (EUR)
sw_per_car = loss_lo * 1e6 / dl_lo                                # software cost per delivered car (EUR)
print(f"R&D embedded in one entry car: EUR {rd_per_car:,.0f}  |  CARIAD loss per delivered car: EUR {sw_per_car:,.0

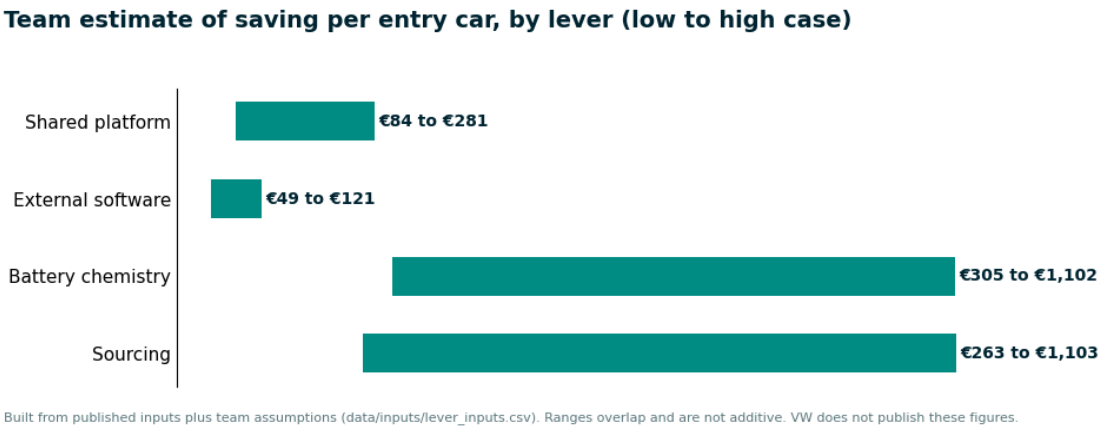
levers = pd.DataFrame([
```

```
    ("Shared platform",    rd_lo / 100 * pr_lo * ps_lo / 100 * cu_lo / 100, rd_hi / 100 * pr_hi * ps_hi / 100 * cu_h
    ("External software", loss_lo * 1e6 / dl_hi * av_lo / 100,              loss_hi * 1e6 / dl_lo * av_hi / 100),
    ("Battery chemistry", kwh_lo * pk_lo * br_lo / 100 / fx_hi,           kwh_hi * pk_hi * br_hi / 100 / fx_lo),
    ("Sourcing",          sp_lo * ug_lo / 100 * sc_lo / 100,               sp_hi * ug_hi / 100 * sc_hi / 100),
], columns=["lever", "low_eur_per_car", "high_eur_per_car"]).round(0)
levers.to_csv(EXHIBITS / "lever_sizing.csv", index=False)
display(levers)

ready = levers.dropna()
if ready.empty:
    print("No lever has all its inputs yet: fill lever_inputs above and re-run.")
else:
    r = ready.iloc[::-1].reset_index(drop=True)
    fig, ax = plt.subplots(figsize=(11, 3.6))
    ax.barh(r["lever"], r["high_eur_per_car"] - r["low_eur_per_car"], left=r["low_eur_per_car"], height=0.5, color=
    for i, (lo, hi) in enumerate(zip(r["low_eur_per_car"], r["high_eur_per_car"])):
        ax.text(hi + 5, i, f"€{lo:,.0f} to €{hi:,.0f}", va="center", fontweight="bold", color=INK)
    ax.set_xlim(0, r["high_eur_per_car"].max() * 1.4)
    ax.set_xticks([])
    ax.tick_params(axis="y", length=0, labelsize=11)
    hide_spines(ax, ("top", "right", "bottom"))
    finish(fig, "Team estimate of saving per entry car, by lever (low to high case)",
           "Built from published inputs plus team assumptions (data/inputs/lever_inputs.csv). Ranges overlap and ar
           "VW does not publish these figures.",
           "07_lever_ranges.png")
```

```
R&D embedded in one entry car: EUR 1,407  |  CARIAD loss per delivered car: EUR 243
```

	lever	low_eur_per_car	high_eur_per_car
0	Shared platform	84.0	281.0
1	External software	49.0	121.0
2	Battery chemistry	305.0	1102.0
3	Sourcing	263.0	1103.0



3.6 Slide 5: estimate for "add a unit-cost program"

Combines the four lever ranges from 3.5 into one program-level estimate. The levers overlap (a cheaper battery changes platform cost), so the sum is cut by an **overlap haircut**. The per-car saving is then scaled by the number of entry cars a year and set against VW Passenger Cars revenue to give margin points, and against the 4.1-point gap to the 6.5% target. The **investment ceiling** is not a forecast: it is the most the program could cost and still pay back within the stated years, a test for the phase gates.

All assumption rows are the team's judgement and can be changed; sourced rows carry their page.

In [12]:
```
program_inputs = pd.DataFrame([
    # input,                                  low,       high,      unit,     type,          source
    ("overlap between levers (haircut)",      20.0,      30.0,      "%",      "assumption", "Team judgement
    ("entry cars a year in the program",      300000.0,  500000.0,  "cars",   "assumption", "Team judgement
    ("VW Passenger Cars sales revenue 2025",  86600.0,   86600.0,   "EUR m",  "sourced",    f"{AR25} p457")
    ("years of full run-rate savings to 2030", 4.0,      4.0,       "years",  "assumption", "Team judgement
], columns=["input", "low", "high", "unit", "type", "source"])
program_inputs.to_csv(INPUTS / "program_inputs.csv", index=False)
display(program_inputs)
```

	input	low	high	unit	type	source
0	overlap between levers (haircut)	20.0	30.0	%	assumption	Team judgement: levers overlap, so the sum ove...
1	entry cars a year in the program	300000.0	500000.0	cars	assumption	Team judgement; four models on MEB+ (Annual Re...
2	VW Passenger Cars sales revenue 2025	86600.0	86600.0	EUR m	sourced	VW Annual Report 2025 (PDF page) p457
3	years of full run-rate savings to 2030	4.0	4.0	years	assumption	Team judgement: full run-rate 2027 to 2030

In [13]:
```
def pget(name):
    r = program_inputs[program_inputs["input"].str.startswith(name)].iloc[0]
    return float(r["low"]), float(r["high"])

ov_lo, ov_hi = pget("overlap")
n_lo, n_hi = pget("entry cars")
rev, _ = pget("VW Passenger Cars sales revenue")
yrs, _ = pget("years of full")

L = levers.dropna()
per_car_lo = L["low_eur_per_car"].sum() * (1 - ov_hi / 100)       # smallest sum, largest overlap
per_car_hi = L["high_eur_per_car"].sum() * (1 - ov_lo / 100)      # largest sum, smallest overlap
annual_lo = per_car_lo * n_lo / 1e6                                # EUR m a year
annual_hi = per_car_hi * n_hi / 1e6
pts_lo, pts_hi = annual_lo / rev * 100, annual_hi / rev * 100      # margin points on the brand
gap = target - vw
program = pd.DataFrame([
    ("Levers sized", ", ".join(L["lever"]), ""),
    ("Saving per entry car (EUR)", f"{per_car_lo:,.0f} to {per_car_hi:,.0f}", "sum of lever ranges less overlap hai
    ("Saving per entry car, % of entry price", f"{per_car_lo / vw_price * 100:.1f}% to {per_car_hi / vw_price * 100
    ("Annual saving (EUR m)", f"{annual_lo:,.0f} to {annual_hi:,.0f}", "x entry cars a year"),
    ("Margin points on VW Passenger Cars", f"{pts_lo:.2f} to {pts_hi:.2f}", f"annual saving / EUR {rev:,.0f} m reve
    ("Share of the 4.1-point gap closed", f"{pts_lo / gap * 100:.0f}% to {pts_hi / gap * 100:.0f}%", f"gap = {targe
    ("Investment ceiling for payback (EUR bn)", f"{annual_lo * yrs / 1000:.1f} to {annual_hi * yrs / 1000:.1f}", f"
], columns=["measure", "estimate", "basis"])
program.to_csv(EXHIBITS / "program_estimate.csv", index=False)
display(program)
program_nums = dict(per_car_lo=per_car_lo, per_car_hi=per_car_hi, annual_lo=annual_lo, annual_hi=annual_hi,
                    pts_lo=pts_lo, pts_hi=pts_hi, gap_lo=pts_lo / gap, gap_hi=pts_hi / gap,
                    ceil_lo=annual_lo * yrs / 1000, ceil_hi=annual_hi * yrs / 1000)
```

	measure	estimate	basis
0	Levers sized	Shared platform, External software, Battery ch...	
1	Saving per entry car (EUR)	491 to 2,086	sum of lever ranges less overlap haircut
2	Saving per entry car, % of entry price	2.3% to 9.9%	of the ID. Polo ex-VAT price
3	Annual saving (EUR m)	147 to 1,043	x entry cars a year
4	Margin points on VW Passenger Cars	0.17 to 1.20	annual saving / EUR 86,600 m revenue
5	Share of the 4.1-point gap closed	4% to 29%	gap = 6.5 - 2.4 points
6	Investment ceiling for payback (EUR bn)	0.6 to 4.2	annual saving x 4 years; a gate test, not a fo...

4. Numbers used in the talk

Every figure quoted on a slide, calculated from the objects above and saved as `data/exhibits/key_numbers.csv` so each number can be traced to its source.

In [14]:
```
latest_3m = float(d.values[-1])
COMEXT = "Eurostat Comext DS-045409, CN8 87038010/87038090, cleaned"
rows = [
    ("S1", "VW Passenger Cars operating margin, H1 2026 (%)", vw, SRC_CORE),
    ("S1", "VW Passenger Cars margin before special items, H1 2026 (%)", vw_adj, SRC_CORE),
    ("S1", "VW brand margin target (%)", target, SRC_TARGET),
    ("S1", "Gap to target, reported (points)", target - vw, "Calculated"),
    ("S1", "Gap to target, before special items (points)", target - vw_adj, "Calculated"),
    ("S1", "Skoda operating margin, H1 2026 (%)", skoda, SRC_CORE),
    ("S2", "Cars a month, 12 months before duties", units_avg["Before"], COMEXT),
    ("S2", "Cars a month, Nov 2024 to Dec 2025", units_avg["After"], COMEXT),
    ("S2", "Cars a month, 2026 to latest month", units_avg["2026"], COMEXT),
    ("S2", "Change after duties vs before", d_after, "Calculated"),
    ("S2", "Change in 2026 vs before", d_latest, "Calculated"),
    ("S3", "Average customs value per car, before (EUR)", value_avg["Before"], COMEXT),
    ("S3", "Average customs value per car, after (EUR)", value_avg["After"], COMEXT),
    ("S3", "Average customs value per car, 2026 (EUR)", value_avg["2026"], COMEXT),
```

```
    ("S3", "Change in value per car, 2026 vs before", chg, "Calculated"),
    ("S3", "Latest 3-month customs value per car (EUR)", latest_3m, COMEXT),
    ("S3", "Latest 3-month value with 27% duties (EUR)", float(v3_duty[last]), "Calculated: value x (1 + 10% + 17%)
]
for _, r in prices.dropna(subset=["list_price_eur"]).iterrows():
    rows.append(("S3", f"{r['maker']} {r['model']} list price ex-VAT (EUR)", float(r["ex_vat"]), r["source"]))
rows.append(("S6", "Group operating margin, H1 2026 (%)", margin, SRC_GROUP))
for _, r in sav.iterrows():
    rows.append(("S6", f"{r['item']} (EUR bn, {r['basis']})", float(r["eur_bn"]), r["source"]))

for _, r in levers.dropna().iterrows():
    rows.append(("S7", f"{r['lever']}: saving per car, low case (EUR)", float(r["low_eur_per_car"]), "Team estimate
    rows.append(("S7", f"{r['lever']}: saving per car, high case (EUR)", float(r["high_eur_per_car"]), "Team estima

PROG_SRC = "Team estimate: lever ranges (3.5) less overlap haircut; assumptions in data/inputs/program_inputs.csv"
for label, key in [("Program saving per entry car, low (EUR)", "per_car_lo"), ("Program saving per entry car, high
                   ("Program annual saving, low (EUR m)", "annual_lo"), ("Program annual saving, high (EUR m)", "an
                   ("Margin points on VW Passenger Cars, low", "pts_lo"), ("Margin points on VW Passenger Cars, hig
                   ("Investment ceiling for payback, low (EUR bn)", "ceil_lo"), ("Investment ceiling for payback, h
    rows.append(("S5", label, float(program_nums[key]), PROG_SRC))

key_numbers = pd.DataFrame(rows, columns=["slide", "figure", "value", "source"])
key_numbers["value"] = key_numbers["value"].round(3)
key_numbers.to_csv(EXHIBITS / "key_numbers.csv", index=False)
display(key_numbers)
print("Files in", EXHIBITS, ":", sorted(p.name for p in EXHIBITS.iterdir()))
```

	slide	figure	value	source
0	S1	VW Passenger Cars operating margin, H1 2026 (%)	2.400	VW press release no. 68/2026, Brand Group Core...
1	S1	VW Passenger Cars margin before special items,...	3.800	VW press release no. 68/2026, Brand Group Core...
2	S1	VW brand margin target (%)	6.500	VW newsroom, 'ACCELERATE FORWARD | Road to 6.5...
3	S1	Gap to target, reported (points)	4.100	Calculated
4	S1	Gap to target, before special items (points)	2.700	Calculated
5	S1	Skoda operating margin, H1 2026 (%)	8.500	VW press release no. 68/2026, Brand Group Core...
6	S2	Cars a month, 12 months before duties	35648.250	Eurostat Comext DS-045409, CN8 87038010/870380...
7	S2	Cars a month, Nov 2024 to Dec 2025	28518.071	Eurostat Comext DS-045409, CN8 87038010/870380...
8	S2	Cars a month, 2026 to latest month	48394.875	Eurostat Comext DS-045409, CN8 87038010/870380...
9	S2	Change after duties vs before	-0.200	Calculated
10	S2	Change in 2026 vs before	0.358	Calculated
11	S3	Average customs value per car, before (EUR)	22018.046	Eurostat Comext DS-045409, CN8 87038010/870380...
12	S3	Average customs value per car, after (EUR)	17531.245	Eurostat Comext DS-045409, CN8 87038010/870380...
13	S3	Average customs value per car, 2026 (EUR)	15592.963	Eurostat Comext DS-045409, CN8 87038010/870380...
14	S3	Change in value per car, 2026 vs before	-0.292	Calculated
15	S3	Latest 3-month customs value per car (EUR)	15508.264	Eurostat Comext DS-045409, CN8 87038010/870380...
16	S3	Latest 3-month value with 27% duties (EUR)	19695.495	Calculated: value x (1 + 10% + 17%)
17	S3	VW ID. Polo Trend list price ex-VAT (EUR)	21004.202	https://www.volkswagen.de/de/modelle.html/__ap...
18	S3	BYD Dolphin Surf Active list price ex-VAT (EUR)	19739.496	https://www.byd.com/de/konfiguration/dolphin-surf
19	S6	Group operating margin, H1 2026 (%)	3.800	VW Group press release no. 69/2026 (24 Jul 2026)
20	S6	Savings in 2024 (EUR bn, per year)	15.000	electrive, 17 Feb 2026 (press report, not a VW...
21	S6	Savings in 2025 (expected) (EUR bn, per year)	18.000	electrive, 17 Feb 2026 (press report, not a VW...
22	S6	Labour costs (EUR bn, per year by 2030)	1.500	electrive, 18 May 2026 (press report quoting V...
23	S6	Production steering, Core group (EUR bn, cumul...	1.000	VW press release no. 68/2026 (24 Jul 2026)
24	S7	Shared platform: saving per car, low case (EUR)	84.000	Team estimate: published inputs plus labelled ...
25	S7	Shared platform: saving per car, high case (EUR)	281.000	Team estimate: published inputs plus labelled ...
26	S7	External software: saving per car, low case (EUR)	49.000	Team estimate: published inputs plus labelled ...
27	S7	External software: saving per car, high case (...	121.000	Team estimate: published inputs plus labelled ...
28	S7	Battery chemistry: saving per car, low case (EUR)	305.000	Team estimate: published inputs plus labelled ...
29	S7	Battery chemistry: saving per car, high case (...	1102.000	Team estimate: published inputs plus labelled ...
30	S7	Sourcing: saving per car, low case (EUR)	263.000	Team estimate: published inputs plus labelled ...
31	S7	Sourcing: saving per car, high case (EUR)	1103.000	Team estimate: published inputs plus labelled ...
32	S5	Program saving per entry car, low (EUR)	490.700	Team estimate: lever ranges (3.5) less overlap...
33	S5	Program saving per entry car, high (EUR)	2085.600	Team estimate: lever ranges (3.5) less overlap...
34	S5	Program annual saving, low (EUR m)	147.210	Team estimate: lever ranges (3.5) less overlap...
35	S5	Program annual saving, high (EUR m)	1042.800	Team estimate: lever ranges (3.5) less overlap...
36	S5	Margin points on VW Passenger Cars, low	0.170	Team estimate: lever ranges (3.5) less overlap...
37	S5	Margin points on VW Passenger Cars, high	1.204	Team estimate: lever ranges (3.5) less overlap...
38	S5	Investment ceiling for payback, low (EUR bn)	0.589	Team estimate: lever ranges (3.5) less overlap...
39	S5	Investment ceiling for payback, high (EUR bn)	4.171	Team estimate: lever ranges (3.5) less overlap...

```
Files in data/exhibits : ['01_margin_gap.png', '01_margin_gap_slide.png', '02_import_trend.png', '02_import_trend_sl
ide.png', '03_price_trend.png', '03_price_trend_slide.png', '06_savings_vs_margin.png', '06_savings_vs_margin_slide.
png', '07_lever_ranges.png', '07_lever_ranges_slide.png', '09_savings_vs_margin.png', 'key_numbers.csv', 'lever_sizi
ng.csv', 'program_estimate.csv']
```